

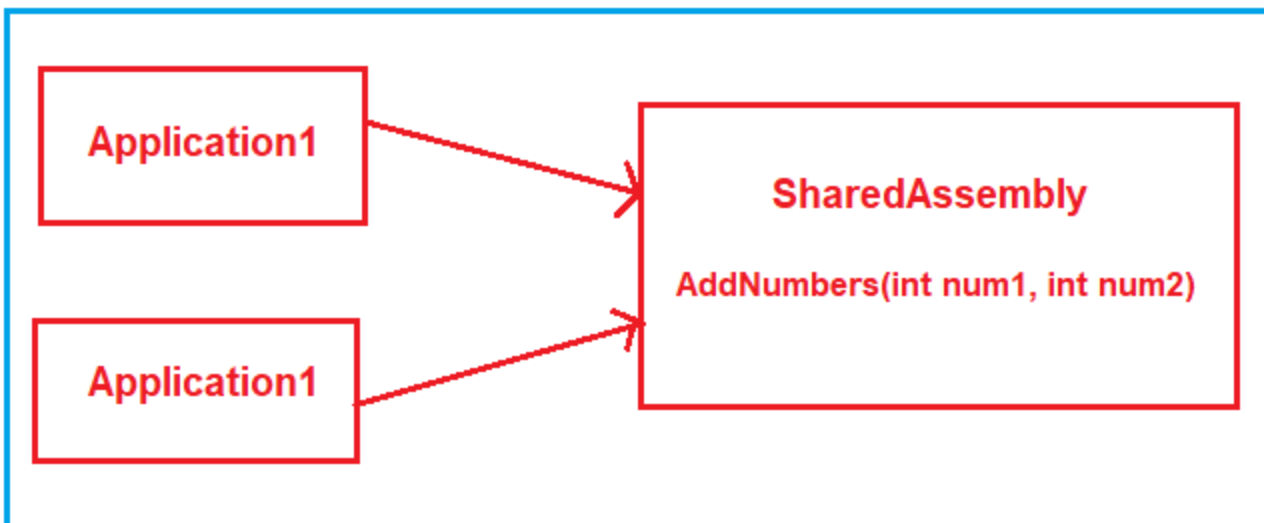
DLL Hell Problem and Solution in .NET Framework

In this article, I am going to discuss **DLL Hell Problem and Solution in .NET Framework** with Examples. Please read our previous article where we discussed [How to Install an Assembly into GAC in .NET Framework](#).

What is DLL Hell Problem in .NET Framework?

DLL Hell is a Problem in .NET Framework in which two different Applications share a common assembly, and If one Application changes the common assembly, and if the changes are not backward compatible, then it will cause the other Application to crash. Let us understand this with an example.

Let say we have two Applications i.e. Application1 and Application2 and both these Applications shares a common Assembly, let us call that common Assembly as **SharedAssembly** (one Class Library Project) and consume the operations of the Common Assembly. Let us assume the Shared Assembly Has one method called **AddNumbers(int num1, int num2)** and this method is going to be consumed by both applications. For a better understanding, please have a look at the following diagram.



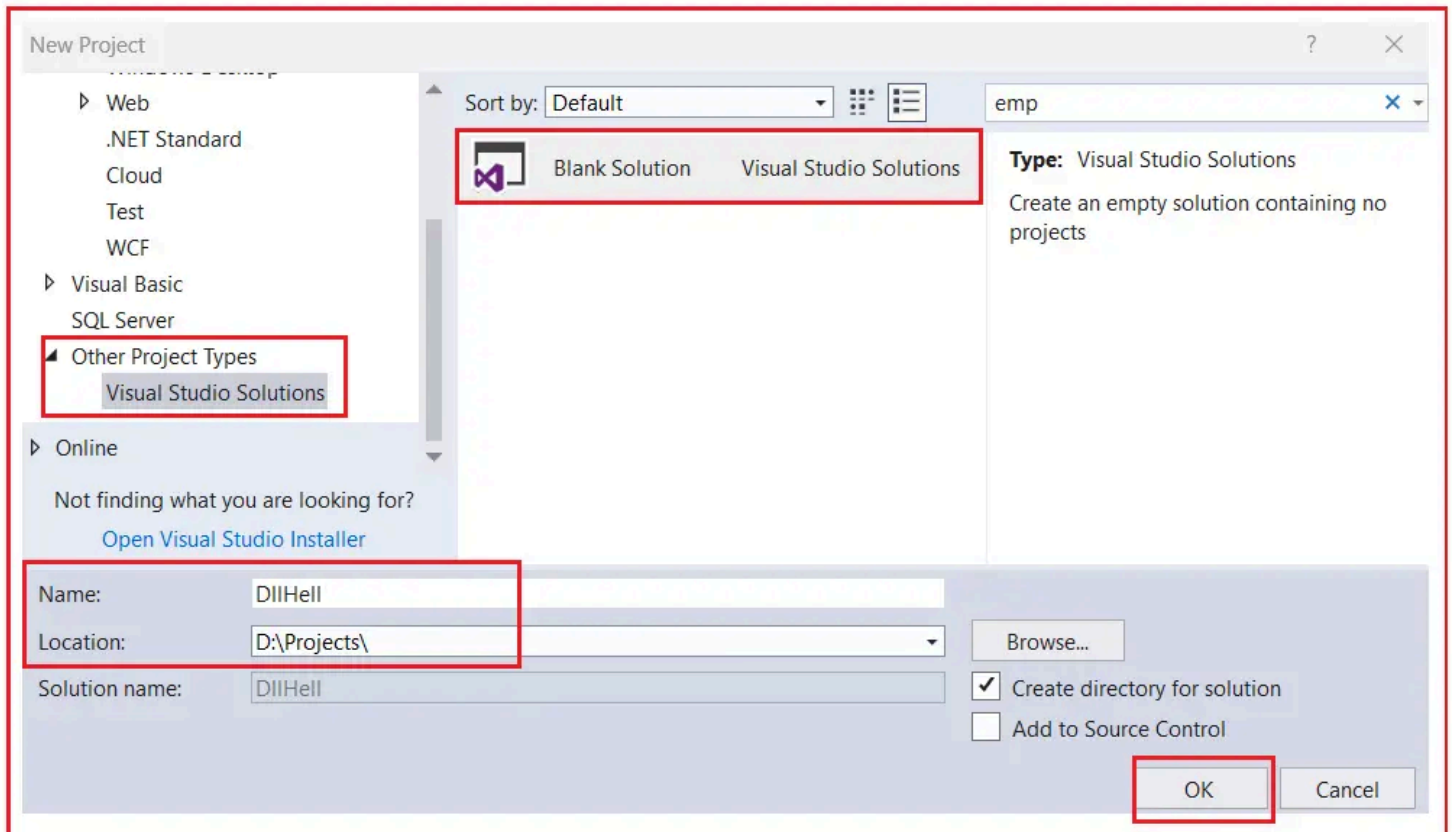
Both Applications are working well. But After some time **Application2** decides to change its application requirements. So during the changes in Application2, it modifies **AddNumbers(int num1, int num2)** method to **AddNumbers(int num1, int num2, int num3)** in **SharedAssembly** assembly Class Library Project. So after this change, **Application2** runs fine as it consumes **AddNumbers(int num1, int num2, int num3)** which is in Shared Assembly but **Application1** crashes because **Application1** still using the **AddNumbers(int num1, int num2)** function. This situation is called as DLL Hell Problem in .NET Framework.

Example to Understand DLL Hell Problem in .NET Framework

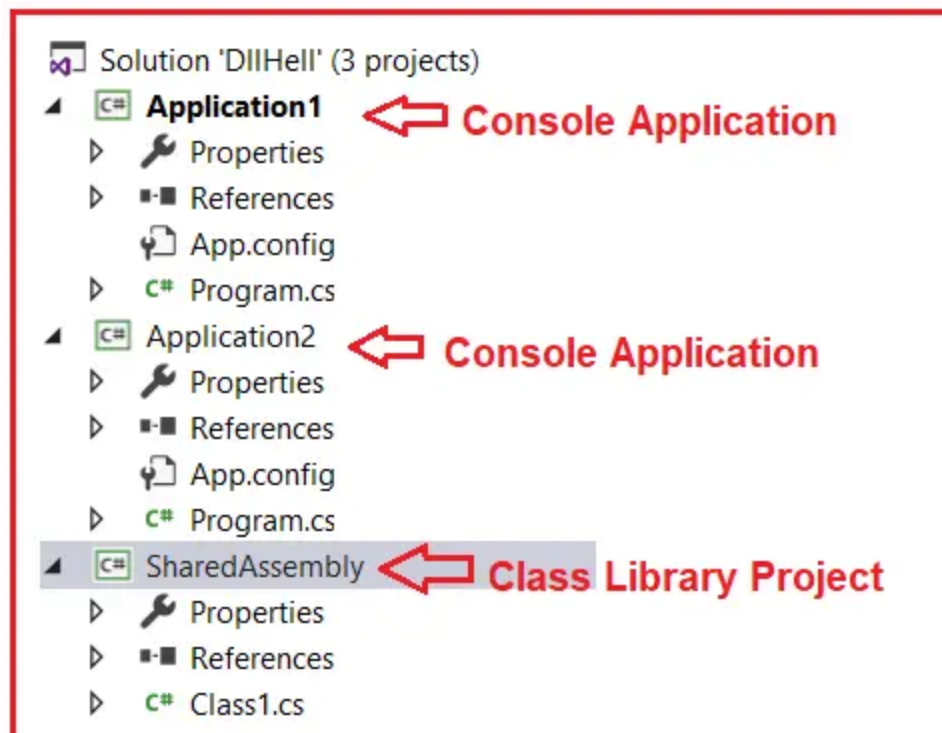
First, we need to create one empty solution and to this Empty solution, we are going to add 3 Projects. Two Console Applications with the name **Application1** and **Application2** and one Class Library Project with the name **SharedAssembly**.

So, open Visual Studio, then select File => New => Project from the context menu which will open the following New Project window. Select Other Project Types, then select Blank Solution, provide the Solution

Name as DIHell and the location where you want to create the Projects, and then click on the OK button as shown in the below image.



Once you click on the OK button, it will create an Empty solution with the name DIHell. To this Empty solution, let us first add two Console Applications with the name **Application1** and **Application2** and one class Library Project with the name **SharedAssembly**. Once you add the projects, your solution should look as shown below.



Modifying SharedAssembly Class Library Project

Next, modify the **Class1.cs** class file code of the **SharedAssembly** Class Library Project as follows. Here, you can see, we are defining one method called **AddNumbers** which takes two integer parameters.

```
namespace SharedAssembly
{
    public class SharedAssemblyClass
    {
        public int AddNumbers(int num1, int num2)
        {
            return num1 + num2;
        }
    }
}
```

Now, build the Class Library Project and once you build the project, one Assembly is going to be generated inside the project Bin=>Debug folder with the name SharedAssembly.dll. Now, the above AddNumbers method of SharedAssembly is going to be consumed by both Application1 and Application2.

Modifying Application1 Console Application:

First, add a reference to the SharedAssembly class library project and then modify the Main method of the Program class as follows. Here, you can see, we are creating an instance of the **SharedAssemblyClass** class and invoking the **AddNumbers** method by passing 10 and 20.

```
using System;
using SharedAssembly;
namespace Application1
{
    class Program
    {
        static void Main(string[] args)
        {
            SharedAssemblyClass obj = new SharedAssemblyClass();
            Console.WriteLine(obj.AddNumbers(10, 20));
            Console.Read();
        }
    }
}
```

Now, make Application1 as the startup project and run the run application and you should get the output as expected i.e. 30.

Modifying Application2 Console Application:

First, add a reference to the SharedAssembly class library project and then modify the Main method of the Program class as follows. Here, you can see, we are creating an instance of the **SharedAssemblyClass** class and invoking the **AddNumbers** method by passing 100 and 200.

```

using System;
using SharedAssembly;
namespace Application2
{
    class Program
    {
        static void Main(string[] args)
        {
            SharedAssemblyClass obj = new SharedAssemblyClass();
            Console.WriteLine(obj.AddNumbers(100, 200));
            Console.Read();
        }
    }
}

```

Now, make Application2 as the startup project and run the run application and you should get the output as expected i.e. 300. In our example the SharedAssembly is shared by two applications and both applications are working fine.

Changing Application1 Requirements:

Now, the business requirements of Application1 changed, now it wants the AddNumbers method to add 3 integer numbers instead of 2 integer numbers. So, for that, we also need to change the AddNumbers method in the SharedAssembly class library project.

So, first, modify the **SharedAssembly** class library project as follows. Here, you can see, we are modifying the AddNumbers method to accept 3 parameters.

```

namespace SharedAssembly
{
    public class SharedAssemblyClass
    {
        public int AddNumbers(int num1, int num2, int num3)
        {
            return num1 + num2 + num3;
        }
    }
}

```

Now, modify the Program.cs class file of the Application1 project as follows. Here, you can see, we are calling the AddNumbers method with 3 parameters.

```

using System;
using SharedAssembly;
namespace Application1
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

        SharedAssemblyClass obj = new SharedAssemblyClass();
        Console.WriteLine(obj.AddNumbers(10, 20, 30));
        Console.Read();
    }
}
}

```

Now, make Application1 as the startup project and run the run application and you should get the output as expected i.e. 60. Now, make Application2 as the startup project and run the run application and you should get the following compile time error.

	Code	Description	Project	File
	CS7036	There is no argument given that corresponds to the required formal parameter 'num3' of 'SharedAssemblyClass.AddNumbers(int, int, int)'	Application2	Program.cs

As you can see the application crashes and this is called DLL Hell Problem in .NET Framework.

How to Overcome DLL Hell Problem in .NET Framework?

In order to overcome the DLL Hell Problem in .NET Framework, we need to sign the shared assemblies with a strong name. With Strongly Named Assembly, we can put different versions of the same assemblies into GAC (Global Assembly cache). So, if Application1 did any breaking change, the change will be in a different version of Shared Assembly, so it will not affect Application1.

GAC stands for Global Assembly Cache and it contains only the Strong Named Assemblies. An assembly is said to be strongly named when it contains the following properties

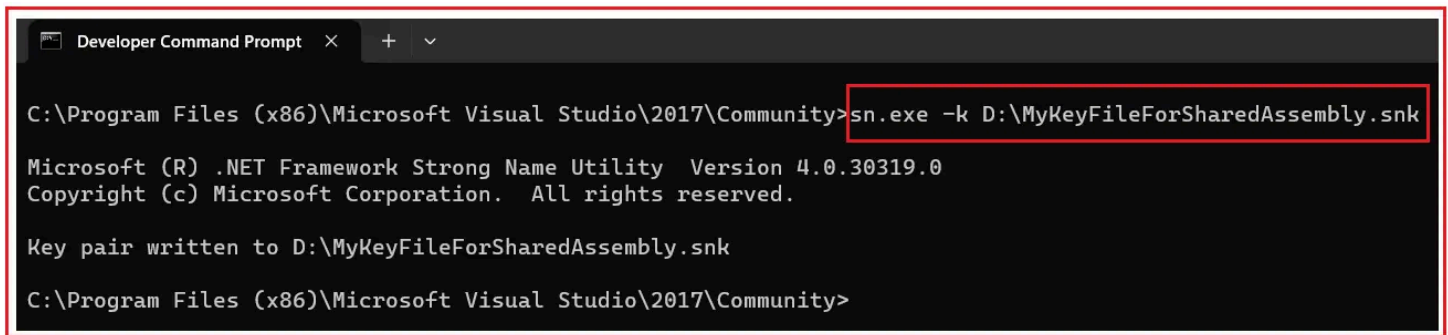
1. **The Assembly Name.**
2. **Version Number.**
3. **The Assembly should have been signed with the Private/Public Key Pair.**

The Global Assembly Cache stores assemblies specifically designated to be shared by several applications on the computer.

How to Make the Assembly a Strong Name Assembly?

To make an assembly strongly named, we need to sign our assembly with a Private and Public Key Pair and we have already discussed this in our previous articles.

In the .NET Framework, we have a tool called Strong Naming Tool (sn.exe) and we can use this (sn.exe) tool to generate the Private Public Key pair. Again, in order to use this tool, we need to use the Developer Command Prompt for Visual Studio. So, open Developer Command Prompt for Visual Studio in Administrator mode and then type **sn.exe -k D:\MyKeyFileForSharedAssembly.snk** and press enter button as shown in the below image.



```
Developer Command Prompt x + v
C:\Program Files (x86)\Microsoft Visual Studio\2017\Community>sn.exe -k D:\MyKeyFileForSharedAssembly.snk
Microsoft (R) .NET Framework Strong Name Utility Version 4.0.30319.0
Copyright (c) Microsoft Corporation. All rights reserved.

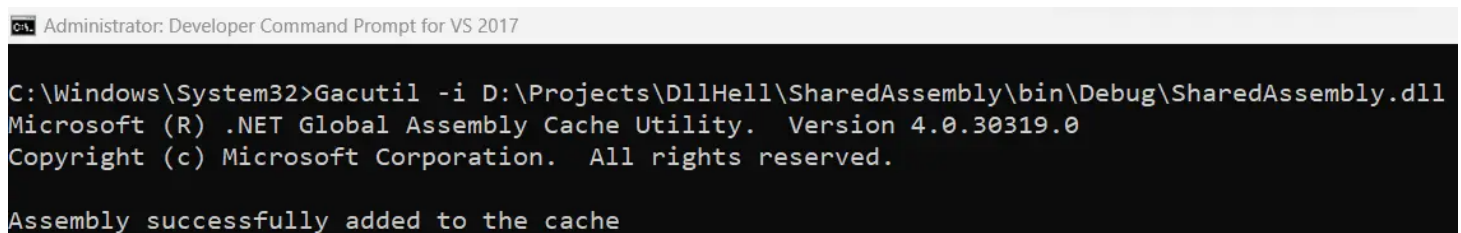
Key pair written to D:\MyKeyFileForSharedAssembly.snk
C:\Program Files (x86)\Microsoft Visual Studio\2017\Community>
```

Once you type the required command and press enter button, the key file with the name **MyKeyFileForSharedAssembly.snk** should be generated in the D: Drive.

Once you generated the Key file, then you need to use the **AssemblyKeyFile** attribute of the **AssemblyInfo** class to sign the assembly with a strong name. So, open the **AssemblyInfo** class file of the **SharedAssembly** class library project, and then to the constructor of the **AssemblyKeyFile** attribute, pass the path of the key file that contains the private and public keys as shown below. You can find the **AssemblyInfo** class within the properties folder of your project.

[assembly: AssemblyKeyFile(@"D:\MyKeyFileForSharedAssembly.snk")]

So, with this change build the **SharedAssembly** class library project and once the build success, our assembly becomes a strongly named assembly. Now, let us install the **SharedAssembly.dll** to GAC. So, open Visual Studio Developer Command Prompt in Administrator mode and then type **Gacutil -i D:\Projects\DllHell\SharedAssembly\bin\Debug\SharedAssembly.dll** command and press the enter button as shown in the below image.



```
Administrator: Developer Command Prompt for VS 2017
C:\Windows\System32>Gacutil -i D:\Projects\DllHell\SharedAssembly\bin\Debug\SharedAssembly.dll
Microsoft (R) .NET Global Assembly Cache Utility. Version 4.0.30319.0
Copyright (c) Microsoft Corporation. All rights reserved.

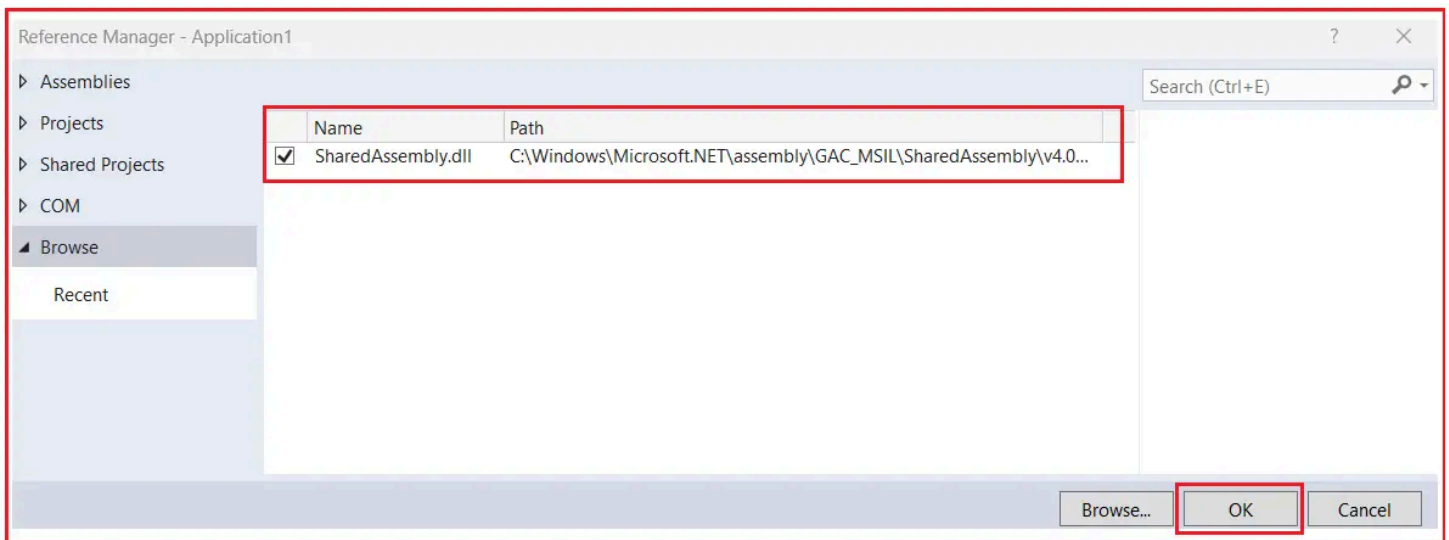
Assembly successfully added to the cache
```

Here, you can see, we are getting the message saying Assembly successfully added to the cache. You can find the assembly in the following location.

C:\Windows\Microsoft.NET\assembly\GAC_MSIL\SharedAssembly\v4.0_1.0.0.0__61124f96c8127571

Use the above Assembly in both Applications:

Now, instead of referring to the SharedAssembly class library project from both the Console Applications, we need to refer to the **SharedAssembly** from GAC. So, first, remove the **SharedAssembly** reference which is currently pointing to the **SharedAssembly** class library project, and then add the **SharedAssembly** from GAC as shown in the below image.



Do the same thing for Application 2. Now, modify the Program class file of Application1 Console Application as follows and run the application and you will get the output as expected.

```
using System;
using SharedAssembly;
namespace Application1
{
    class Program
    {
        static void Main(string[] args)
        {
            SharedAssemblyClass obj = new SharedAssemblyClass();
            Console.WriteLine(obj.AddNumbers(10, 20, 30));
            Console.Read();
        }
    }
}
```

Now, modify the Program class file of Application2 Console Application as follows and run the application and you will get the output as expected.

```
using System;
using SharedAssembly;
namespace Application2
{
    class Program
    {
        static void Main(string[] args)
        {
            SharedAssemblyClass obj = new SharedAssemblyClass();
            Console.WriteLine(obj.AddNumbers(100, 200, 300));
            Console.Read();
        }
    }
}
```

Changing Requirements:

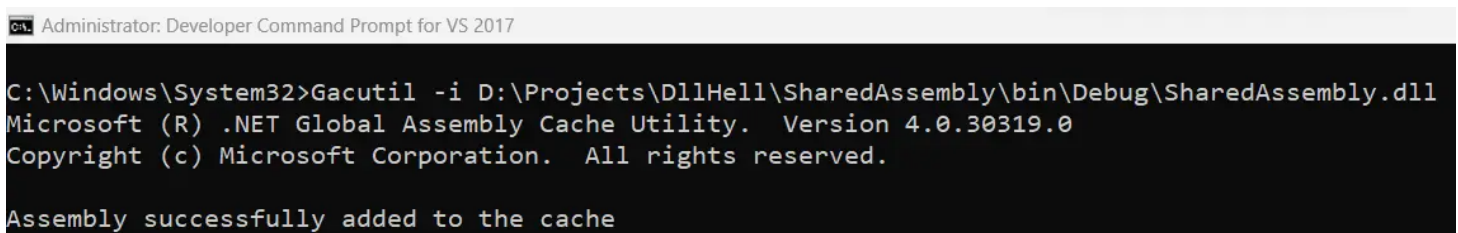
Now, Application1 requirements have changed. Instead of adding 3 integer numbers, application 1 wants to add 2 integer numbers. For that, Application1 needs changes in the SharedAssembly class library project. So, open the SharedAssembly class library project and then modify the Class1.cs class file as follows:

```
namespace SharedAssembly
{
    public class SharedAssemblyClass
    {
        public int AddNumbers(int num1, int num2)
        {
            return num1 + num2;
        }
    }
}
```

Next, as we have made some changes to the class library project, let us update the AssemblyVersion attribute of the AssemblyInfo class of the SharedAssembly class library project. Currently, the value of the AssemblyVersion attribute is “1.0.0.0”, let us update this value to “2.0.0.0” as follows.

[assembly: AssemblyVersion(“2.0.0.0”)]

Now, build the SharedAssembly class library project and again installed the assembly into GAC by using the command **Gacutil -i D:\Projects\DllHell\SharedAssembly\bin\Debug\SharedAssembly.dll** in Visual Studio Command Prompt in Administrator mode as shown in the below image.

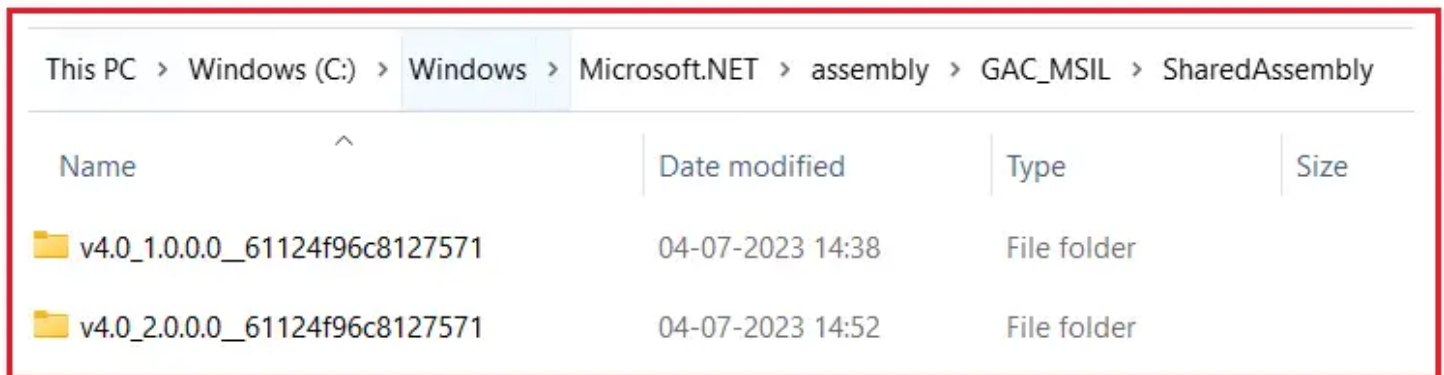


```
Administrator: Developer Command Prompt for VS 2017

C:\Windows\System32>Gacutil -i D:\Projects\DllHell\SharedAssembly\bin\Debug\SharedAssembly.dll
Microsoft (R) .NET Global Assembly Cache Utility. Version 4.0.30319.0
Copyright (c) Microsoft Corporation. All rights reserved.

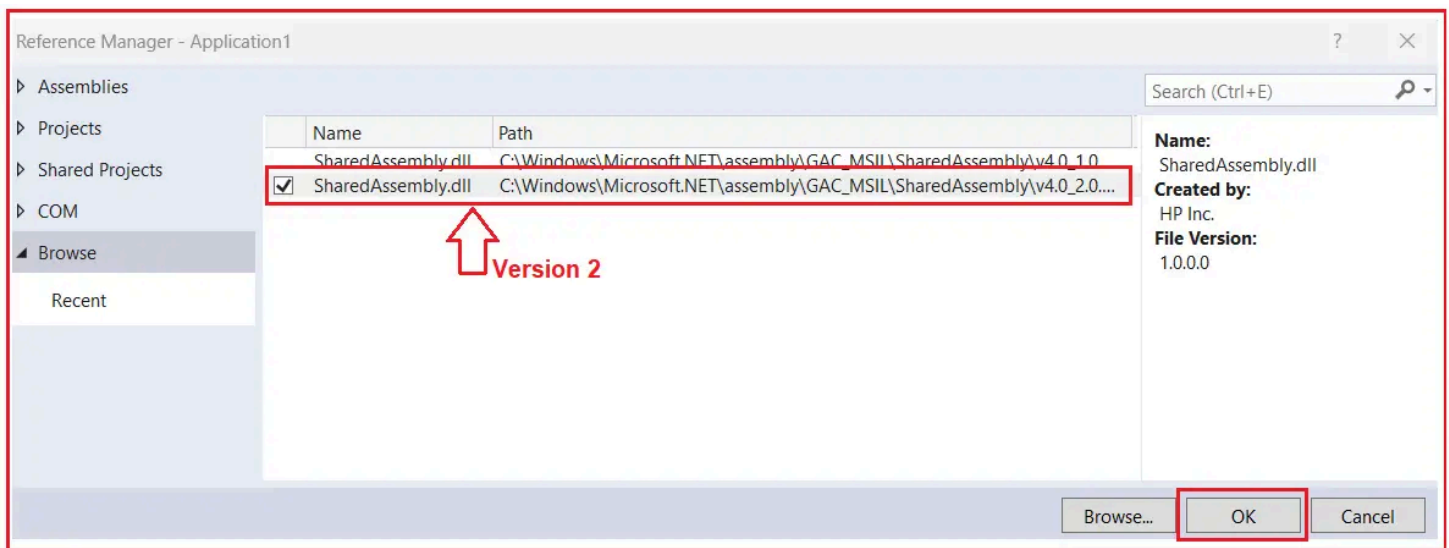
Assembly successfully added to the cache
```

Now, if you go to the GAC, then you will see that two versions of the SharedAssembly are available as shown in the below image.



This PC > Windows (C:) > Windows > Microsoft.NET > assembly > GAC_MSIL > SharedAssembly			
Name	Date modified	Type	Size
 v4.0_1.0.0.0_61124f96c8127571	04-07-2023 14:38	File folder	
 v4.0_2.0.0.0_61124f96c8127571	04-07-2023 14:52	File folder	

Now, from Application1, first, remove the SharedAssembly reference and then add the SharedAssembly reference whose version is 2 as shown in the below image.



Now, modify the Program class code of the Application1 console application as follows. Here, we are invoking the method by passing two integer numbers and it is working fine. Make Application1 the startup project and run the application.

```
using System;
using SharedAssembly;
namespace Application1
{
    class Program
    {
        static void Main(string[] args)
        {
            SharedAssemblyClass obj = new SharedAssemblyClass();
            Console.WriteLine(obj.AddNumbers(10, 20));
            Console.Read();
        }
    }
}
```

Now, make Application2 as the startup project and run the application and you will see that it is also working as expected. Now, Application1 uses SharedAssembly Version 2 and Application2 uses SharedAssembly Version 1. So, in this way, by using Strong Named Assembly we can overcome the DLL Hell Problem in .NET Framework.

In the next article, I am going to discuss **Satellite Assembly in .NET Framework** with Examples. Here, in this article, I try to explain **DLL Hell Problems and How to Overcome DLL Hell Problems in .NET Framework** with Examples. I hope you enjoy this DLL Hell Problem and Solution in .NET Framework article